EFFAT UNIVERSITY
Department of Computer Science

AI Course — CS4086

LAB TUTORIAL

Knowledge Representation & Propositional Logic in Python

Enhanced Edition — Beginner-Friendly with Plain-English Explanations

---

Chapter: Knowledge-Based AI — Logical Agents

`Lab Files: logic.py | harry.py | clue.py | mastermind.py | puzzle.py`

Estimated Duration: **2 Hours**

Advanced AI — Academic Year 2024–2025

Contents

1 Lab Overview **2**

2 Key Concepts Before You Code **2**
- 2.1 What is a Proposition? . . . 3
- 2.2 What is a Model? . . . 3
- 2.3 What is a Knowledge Base? . . . 3

3 The Five Logic Connectives — Plain English Guide **4**
- 3.1 NOT — Flipping the Truth Value . . . 4
- 3.2 AND — All Must Be True . . . 4
- 3.3 OR — At Least One Must Be True . . . 5
- 3.4 Implication ($P \to Q$) — "If... Then..." . . . 5
- 3.5 Biconditional ($P \leftrightarrow Q$) — "If and Only If" . . . 6

4 The Logic Engine: `logic.py` **7**
- 4.1 The `Sentence` Base Class . . . 7
- 4.2 The Six Classes — How Each One Works . . . 8
  - 4.2.1 Symbol — The Simplest Building Block . . . 8
  - 4.2.2 Not, And, Or — Composing Sentences . . . 8
  - 4.2.3 Implication — The "If... Then..." Rule . . . 9
  - 4.2.4 Biconditional — The "Exact Match" Rule . . . 9
- 4.3 The `model_check()` Algorithm — The Reasoning Engine . . . 9

5 `harry.py` — Simple Logical Deduction **11**
- 5.1 The Scenario (Read This First!) . . . 11
- 5.2 The Code . . . 11
- 5.3 Step-by-Step Reasoning — How the AI Thinks . . . 11
- 5.4 Key Concepts Illustrated . . . 12

6 `clue.py` — Cluedo Reasoning with Uncertainty **12**
- 6.1 Variables and Setup . . . 12
- 6.2 The `check_knowledge()` Function — Three-Way Output . . . 13
- 6.3 Building the Knowledge Base Layer by Layer . . . 14
- 6.4 Reasoning Through the Clue Puzzle — Step by Step . . . 15

7 `mastermind.py` — Encoding Game Feedback as Logic **15**
- 7.1 The Symbol Naming Scheme . . . 15
- 7.2 Structural Constraints — The Game Rules . . . 15
- 7.3 Encoding Guess Feedback — The Tricky Part . . . 16

8 `puzzle.py` — Harry Potter House Assignment **18**
- 8.1 Variables . . . 18
- 8.2 Constraints and Clues . . . 18
- 8.3 Full Deduction Chain — Step by Step . . . 19

9 Lab Exercises **19**

9.1 Exercise 1 — Truth Table Analysis (Warm-up) 19
9.2 Exercise 2 — Extend `harry.py` 20
9.3 Exercise 3 — New Clue Scenario 20
9.4 Exercise 4 — Add a Third Clue to `mastermind.py` 21
9.5 Exercise 5 — Scalability Analysis (Discussion) 21

10 Summary **22**

1. Lab Overview

Big Picture — What Are We Building?

Imagine you are a detective. You have a notebook (the **knowledge base**) where you write down everything you know as facts and rules. When you need to answer a question, you look through your notebook and **reason** your way to the answer.

That is exactly what a logical AI agent does. Instead of guessing or learning from data, it **applies rules of logic** to reach guaranteed conclusions. In this lab, you will build such agents in Python.

This lab explores how an AI agent can **represent knowledge** about the world and use **logical reasoning** to draw new conclusions. Rather than hard-coding answers, a logical agent builds a knowledge base of facts and rules, then uses an inference engine to determine what must be true.

Learning Objectives

By the end of this lab, students will be able to:

- Understand propositional logic connectives: NOT, AND, OR, Implication, Biconditional
- Build a knowledge base using the Python `logic.py` library
- Use the `model_check()` algorithm to query logical entailment
- Encode real-world puzzle constraints as logical sentences
- Analyze and extend existing logic-based AI programs

Background Reading

Before starting this lab, review the following from your course notes:

- Russell & Norvig: **Chapter 7** — Logical Agents
- Lecture slides: Propositional Logic & Knowledge-Based Agents
- Course notes on Model Checking and Truth Tables

2. Key Concepts Before You Code

Before touching any code, make sure you understand these three ideas clearly. Everything else in this lab builds on them.

2.1 What is a Proposition?

Plain English

A **proposition** is any statement that is either **TRUE** or **FALSE** — it cannot be both, and it cannot be "maybe".

Examples of propositions:

- "It is raining outside." (True or False)
- "Mustard is the murderer." (True or False)
- "Red is in position 0." (True or False)

NOT a proposition: "What time is it?" (a question, not a statement)

In code, each proposition is represented by a `Symbol`:

```
rain = Symbol("rain")    # The proposition "it is raining"
```

2.2 What is a Model?

Plain English

A **model** is one possible snapshot of the world — a complete assignment of TRUE or FALSE to every proposition.

Think of it like filling in a questionnaire where every answer must be Yes or No:

Is it raining?	**YES**
Did Harry meet Hagrid?	**NO**
Did Harry meet Dumbledore?	**YES**

In Python, a model is just a dictionary: `{"rain": True, "hagrid": False, "dumbledore": True}`

2.3 What is a Knowledge Base?

Plain English

A **knowledge base (KB)** is the collection of all the facts and rules you know to be true. It is like the detective's notebook.

The AI only considers worlds (models) that are **consistent with everything in the KB**. Impossible worlds are thrown out.

If your KB says "it is raining" and "if it rains, the ground is wet", then the AI will only consider worlds where both statements hold — so in every world it considers, the ground must be wet.

3. The Five Logic Connectives — Plain English Guide

These five connectives are the grammar of logic. You use them to build complex statements out of simple propositions.

Connective	Symbol	Everyday Meaning	True when...
`Not`	$\neg P$	"It is NOT the case that..."	P is False
`And`	$P \wedge Q$	"Both P and Q"	ALL parts are True
`Or`	$P \vee Q$	"At least one of P or Q"	ANY part is True
`Implication`	$P \rightarrow Q$	"If P, then Q"	P is False, OR Q is True
`Biconditiona`	$P \leftrightarrow Q$	"P exactly when Q"	Both True or both False

3.1 NOT — Flipping the Truth Value

Real-World Analogy

If the statement is "The light is ON", then **NOT** gives you "The light is OFF." NOT simply flips True to False and False to True. That's all it does.

```
rain = Symbol("rain")

# If rain is True, Not(rain) is False
# If rain is False, Not(rain) is True
Not(rain).evaluate({"rain": False})    # Returns: True  (it is NOT raining = True)
Not(rain).evaluate({"rain": True})     # Returns: False (it is NOT raining = False)
```

3.2 AND — All Must Be True

Real-World Analogy

Think of AND like a security door that requires **both** a keycard AND a PIN. If either one is missing, the door stays locked.

```
rain = Symbol("rain")
wet  = Symbol("wet")

# AND: BOTH must be true
```

```python
And(rain, wet).evaluate({"rain": True,  "wet": True})   # True (both OK)
And(rain, wet).evaluate({"rain": True,  "wet": False})  # False (wet is missing)
And(rain, wet).evaluate({"rain": False, "wet": True})   # False (rain is missing)

# Special feature: you can build a knowledge base step by step using .add()
kb = And()          # start empty
kb.add(rain)        # add first fact
kb.add(wet)         # add second fact
# Now kb represents: rain AND wet
```

3.3 OR — At Least One Must Be True

Real-World Analogy

Think of OR like a bus schedule: you can take **Bus A or Bus B**. As long as at least one bus is running, you can get home. Both running is also fine.

```python
Or(rain, wet).evaluate({"rain": False, "wet": True})    # True (wet saves it)
Or(rain, wet).evaluate({"rain": False, "wet": False})   # False (neither is true)
Or(rain, wet).evaluate({"rain": True,  "wet": True})    # True (both fine too)
```

3.4 Implication ($P \to Q$) — "If... Then..."

This is the most commonly misunderstood connective. Study this carefully.

Real-World Analogy

"If it rains, I will bring an umbrella."

This statement is a **PROMISE**. It is only **broken** (= False) when:

- It IS raining **AND** you did NOT bring an umbrella. (You broke your promise.)

In all other cases, the promise is **kept** (= True):

- It rains AND you brought an umbrella — promise kept. (**True**)
- It does NOT rain AND you brought an umbrella anyway — irrelevant, promise kept. (**True**)
- It does NOT rain AND you didn't bring an umbrella — the promise was never triggered. (**True**)

```python
rain  = Symbol("rain")
umbrella = Symbol("umbrella")
rule = Implication(rain, umbrella)   # "If rain, then umbrella"
```

```python

rule.evaluate({"rain": True,  "umbrella": True})    # True  (rain+umbrella: OK)
rule.evaluate({"rain": True,  "umbrella": False})   # False (rained but no umbrella!)
rule.evaluate({"rain": False, "umbrella": True})    # True  (no rain: rule not triggered)
rule.evaluate({"rain": False, "umbrella": False})   # True  (no rain: rule not triggered)
```

Students often think: "If it didn't rain, the implication must be False." **This is WRONG.**

When the condition (P) is False, the implication is **automatically True** — the rule was never triggered, so it was never violated. This is called a *vacuously true* implication.

3.5 Biconditional ($P \leftrightarrow Q$) — "If and Only If"

Real-World Analogy

A light switch and a lamp: the lamp is ON **if and only if** the switch is ON. They always match. If one is True, the other must be True. If one is False, the other must be False.

```python
switch = Symbol("switch")
lamp   = Symbol("lamp")
rule   = Biconditional(switch, lamp)

rule.evaluate({"switch": True,  "lamp": True})     # True  (both ON: match)
rule.evaluate({"switch": False, "lamp": False})    # True  (both OFF: match)
rule.evaluate({"switch": True,  "lamp": False})    # False (mismatch!)
rule.evaluate({"switch": False, "lamp": True})     # False (mismatch!)
```

4. The Logic Engine: `logic.py`

What is logic.py?

`logic.py` is a Python library that gives you ready-made building blocks for propositional logic. Think of it as a LEGO set for logic: `Symbol`, `Not`, `And`, `Or`, `Implication`, and `Biconditional` are your LEGO pieces. You snap them together to build a knowledge base, and then `model_check()` tells you what conclusions you can draw.

All four lab programs depend on `logic.py`. Before studying the puzzles, you must understand this foundation.

4.1 The `Sentence` Base Class

Every logical construct in `logic.py` is a `Sentence`. All classes (Symbol, Not, And, etc.) inherit from it. The base class defines three methods that every sentence must support:

```
class Sentence():
    def evaluate(self, model):    # Q: Is this sentence TRUE in this world?
        raise Exception("nothing to evaluate")

    def formula(self):            # Q: Show me the sentence as readable text
        return ""

    def symbols(self):            # Q: What variables does this sentence use?
        return set()
```

Plain English — What these methods do

- `evaluate(model)` — Given a snapshot of the world (True/False for each variable), is this sentence True or False? *Example: Is "it is raining AND the ground is wet" True in this world?*
- `formula()` — Convert the sentence into a readable string like `(rain AND wet)`.
- `symbols()` — List all the variable names used. *Example: `"rain"` and `"wet"` for the sentence above.*

4.2 The Six Classes — How Each One Works

Class	Symbol	Meaning	`evaluate()` returns True when...
`Symbol`	P	Propositional variable	`model[name]` is `True`
`Not`	$\neg P$	Negation	operand evaluates to `False`
`And`	$P \wedge Q$	Conjunction	**ALL** conjuncts are `True`
`Or`	$P \vee Q$	Disjunction	**ANY** disjunct is `True`
`Implication`	$P \rightarrow Q$	If...then...	P is `False` **OR** Q is `True`
`Bicondition`	$P \leftrightarrow Q$	If and only if	Both `True` or both `False`

4.2.1 Symbol — The Simplest Building Block

A `Symbol` is just a name for a proposition. Its `evaluate()` method simply looks up the name in the model dictionary:

```
rain = Symbol("rain")

# In a world where rain = True:
rain.evaluate({"rain": True})     # Returns: True
rain.symbols()                    # Returns: {"rain"}  -- the variables it uses

# In a world where rain = False:
rain.evaluate({"rain": False})    # Returns: False
```

4.2.2 Not, And, Or — Composing Sentences

These classes take other `Sentence` objects as input and combine them:

```
rain = Symbol("rain")
wet  = Symbol("wet")

#         NOT


Not(rain).evaluate({"rain": False})               # True  (it is NOT raining)

#         AND


And(rain, wet).evaluate({"rain": True, "wet": False})    # False (wet is missing)

# AND has a special .add() method to build a KB one sentence
```

```
    at a time:
kb = And()          # empty KB
kb.add(rain)        # step 1: add "it is raining"
kb.add(wet)         # step 2: add "the ground is wet"
# kb now means: "it is raining AND the ground is wet"

#         OR


Or(rain, wet).evaluate({"rain": False, "wet": True})     # True
    (wet saves it)
```

4.2.3 Implication — The "If... Then..." Rule

The implementation translates the promise into code. Remember: a promise is only broken when the condition is True but the result is False:

```
def evaluate(self, model):
    # The promise is KEPT (True) when:
    # - The condition (antecedent) is FALSE  (rule was never
        triggered), OR
    # - The result (consequent) is TRUE      (rule was
        followed)
    return ((not self.antecedent.evaluate(model))
            or self.consequent.evaluate(model))
```

4.2.4 Biconditional — The "Exact Match" Rule

```
def evaluate(self, model):
    # True when both sides MATCH (both True, or both False)
    left_val  = self.left.evaluate(model)
    right_val = self.right.evaluate(model)
    return ((left_val and right_val)
            or (not left_val and not right_val))
```

4.3 The `model_check()` Algorithm — The Reasoning Engine

Plain English — What does model_check() do?

`model_check(KB, query)` answers the question:

"Does my knowledge base GUARANTEE that the query is true?"

It works by trying **every possible world** (every True/False combination for every variable). For each world where the KB is satisfied, it checks whether the query is also true. If there is even ONE world where the KB is true but the query is false, the answer is **NO**. Otherwise, the answer is **YES**.

```
def model_check(knowledge, query):

    def check_all(knowledge, query, symbols, model):
```

```
        if not symbols:                          # No more variables to assign
            if knowledge.evaluate(model):        # Is the KB satisfied here?
                return query.evaluate(model)     # If yes, query MUST also be true
            return True                          # KB is false here: skip this world

        else:
            remaining = symbols.copy()
            p = remaining.pop()                  # Pick one unassigned variable

            # Try assigning it True, and try assigning it False:
            model_true  = model.copy(); model_true[p]  = True
            model_false = model.copy(); model_false[p] = False

            # The query is entailed only if it holds in BOTH branches
            return (check_all(knowledge, query, remaining, model_true) and
                    check_all(knowledge, query, remaining, model_false))

    symbols = set.union(knowledge.symbols(), query.symbols())
    return check_all(knowledge, query, symbols, dict())
```

Analogy — The Branching Tree

Imagine you have 3 variables: `rain`, `hagrid`, `dumbledore`.

`model_check` builds a **decision tree**:

- Level 1: rain = True? OR rain = False?
- Level 2: hagrid = True? OR hagrid = False?
- Level 3: dumbledore = True? OR dumbledore = False?

This gives $2^3 = 8$ leaf nodes (complete worlds). At each leaf, it checks: "Does the KB hold here?" If yes, it also checks the query. The query is **guaranteed** (entailed) only if it passes in **every leaf where KB holds**.

Entailment — The Key Idea

$KB \models Q$ (read: "KB entails Q") means: **Q is True in every world where KB is True.**

This is different from probability! It is not "Q is probably true." It is "Q is **certainly** true — no exceptions."

With n symbols, the algorithm explores 2^n worlds. This is complete and always correct, but slow for large problems.

5. harry.py — Simple Logical Deduction

What This Program Does

This is the simplest example. It shows how a small set of facts and rules lets the AI reach a **definite conclusion**. Think of it as a beginner logic puzzle where you reason step by step.

5.1 The Scenario (Read This First!)

Harry Potter had a day where exactly one of Hagrid or Dumbledore visited him. We are given the following facts:

- **Rule 1:** If it was NOT raining, Harry went to see Hagrid.
- **Rule 2:** Harry met Hagrid OR Dumbledore (at least one).
- **Rule 3:** Harry did NOT meet BOTH of them (exactly one).
- **Fact:** Harry met Dumbledore.

Question: Was it raining?

5.2 The Code

```python
rain        = Symbol("rain")
hagrid      = Symbol("hagrid")
dumbledore  = Symbol("dumbledore")

knowledge = And(
    Implication(Not(rain), hagrid),      # Rule 1: no rain -> met Hagrid
    Or(hagrid, dumbledore),              # Rule 2: met Hagrid or Dumbledore
    Not(And(hagrid, dumbledore)),        # Rule 3: not both
    dumbledore                           # Fact:   met Dumbledore
)

print(model_check(knowledge, rain))      # Output: True
```

5.3 Step-by-Step Reasoning — How the AI Thinks

Follow the Logic Chain

Step 1 — Start with what we know for certain:
Fact: Dumbledore = **True** (given directly).

Step 2 — Apply Rule 3 (Not Both):

"NOT (hagrid AND dumbledore)" — since Dumbledore is True, if Hagrid were also True, both would be True, which violates Rule 3. So Hagrid = **False**.

Step 3 — Check Rule 2 (Or):
"hagrid OR dumbledore" = False OR True = **True**. Satisfied. Good.

Step 4 — Apply Rule 1 (Contrapositive):
Rule 1 says: "Not rain → Hagrid". We know Hagrid is **False**.
If Not rain were True, then Hagrid would HAVE to be True (by Rule 1). But Hagrid is False. Contradiction!
Therefore: Not rain = **False**, which means Rain = **True**.

Conclusion: It was raining. `model_check` returns **True**.

1. **Fact:** `dumbledore = True` (given directly)
2. **Rule 3 + dumbledore:** `Not(And(hagrid, dumbledore))` ⇒ `hagrid` must be `False`
3. **Rule 2 + Not(hagrid):** `Or(hagrid, dumbledore)` is satisfied only by `dumbledore` — OK
4. **Rule 1 (contrapositive):** If `hagrid` is `False`, then ¬(¬rain) ⇒ `rain = True`
5. **Conclusion:** `rain` is entailed. Output: `True`

5.4 Key Concepts Illustrated

- **Contrapositive reasoning:** "$P \to Q$" is logically the same as "$\neg Q \to \neg P$". We know Hagrid did NOT come, so by the contrapositive of Rule 1, it must have been raining.
- **Exclusive constraint:** Using `Or` (at least one) together with `Not(And(...))` (not both) creates an *exclusive or* — exactly one of the two.
- **The KB is a conjunction:** All four sentences must be simultaneously true. The AI only considers worlds where ALL of them hold.

6. `clue.py` — Cluedo Reasoning with Uncertainty

What This Program Does

This program models the classic board game Cluedo (Clue). The AI must figure out: who did it, in which room, with which weapon? But it doesn't know everything — it has **partial information** and must reason carefully under uncertainty. Some answers will be **YES** (certain), some **MAYBE** (not enough info yet), and some silently **NO** (impossible).

6.1 Variables and Setup

```python
# Three categories: who, where, what
mustard  = Symbol("ColMustard")     # Suspect 1
plum     = Symbol("ProfPlum")       # Suspect 2
scarlet  = Symbol("MsScarlet")      # Suspect 3

ballroom = Symbol("ballroom")       # Room 1
kitchen  = Symbol("kitchen")        # Room 2
library  = Symbol("library")        # Room 3

knife    = Symbol("knife")          # Weapon 1
revolver = Symbol("revolver")       # Weapon 2
wrench   = Symbol("wrench")         # Weapon 3
```

6.2 The check_knowledge() Function — Three-Way Output

Plain English — YES, MAYBE, or (silent) NO

For each variable, the function asks the KB two questions:

1. **"Is this variable definitely TRUE?"** — `model_check(KB, S)`
   If YES: print "S: YES" in green.
2. **"Is this variable definitely FALSE?"** — `model_check(KB, Not(S))`
   If YES: print nothing (it's eliminated).
   If NO to both: print "S: MAYBE" (the KB hasn't decided yet).

```python
def check_knowledge(knowledge):
    for symbol in symbols:
        if model_check(knowledge, symbol):                # Definitely TRUE
            termcolor.cprint(f"{symbol}: YES", "green")
        elif not model_check(knowledge, Not(symbol)):    # Could be True or False
            print(f"{symbol}: MAYBE")
        # If model_check(KB, Not(symbol)) = True: definitely FALSE (not printed)
```

Condition	Output	What it means
`model_check(KB, S) = True`	YES (green)	S is true in every possible world consistent with KB
`model_check(KB, Not(S)) = False`	MAYBE	S could go either way; KB doesn't have enough info
`model_check(KB, Not(S)) = True`	(silent)	S is false in every possible world consistent with KB

6.3 Building the Knowledge Base Layer by Layer

Think of it like narrowing down suspects

You start with "anyone could have done it" and then eliminate possibilities one by one as you learn new information. Each layer of the KB removes more impossible worlds.

Layer 1 — The crime must have a suspect, room, and weapon:

```
knowledge = And(
    Or(mustard, plum, scarlet),        # Someone did it (at least one suspect)
    Or(ballroom, kitchen, library),    # Somewhere (at least one room)
    Or(knife, revolver, wrench)        # With something (at least one weapon)
)
```

Layer 2 — Cards in YOUR hand (you can see them; they are NOT the murder items):

```
# You hold: Colonel Mustard, Kitchen, Revolver
# So these three are definitely NOT the murder answer:
knowledge.add(And(
    Not(mustard), Not(kitchen), Not(revolver)
))
```

Layer 3 — A card shown by another player:

```
# An opponent showed you ONE card from {Scarlet, Library, Wrench}
# You don't know which one -- but at least one of them is in their hand (= not murder)
knowledge.add(Or(
    Not(scarlet), Not(library), Not(wrench)
))
```

When an opponent shows you a card to refute your accusation, you only know that **at least one** of the three cards is theirs. You didn't see which one! So you cannot say "Not(scarlet) AND Not(library) AND Not(wrench)" — that would claim you know all three are innocent. Instead, you say "Not(scarlet) OR Not(library) OR Not(wrench)" — at least one is innocent, but you don't know which.

Layer 4 — Additional cards you've seen:

```
knowledge.add(Not(plum))        # You've seen Professor Plum's card
knowledge.add(Not(ballroom))    # You've seen the ballroom card
```

6.4 Reasoning Through the Clue Puzzle — Step by Step

1. `Not(mustard)` and `Not(plum)` are known ⇒ among the suspects, only `scarlet` remains.
   The Layer 1 rule says at least one suspect must be the murderer ⇒ **MsScarlet: YES**

2. `Not(kitchen)` and `Not(ballroom)` are known ⇒ only `library` remains.
   ⇒ **library: YES**

3. `Not(revolver)` is known ⇒ knife or wrench remain.
   Layer 3 says: Not(scarlet) OR Not(library) OR Not(wrench). Since scarlet and library ARE the murder items (YES), both Not(scarlet) and Not(library) are False in the murder world. So Not(wrench) must be True ⇒ `wrench` is eliminated.
   ⇒ **knife: YES**

7. `mastermind.py` — Encoding Game Feedback as Logic

What This Program Does

Mastermind is a code-breaking game. The AI must figure out the correct order of 4 colors based on feedback from guesses. This is the most complex program — it has **16 symbols** and requires careful encoding of "exactly N positions correct" feedback.

7.1 The Symbol Naming Scheme

Each symbol represents: **"color C is in position P"**. With 4 colors and 4 positions, there are $4 \times 4 = 16$ symbols:

```
colors = ["red", "blue", "green", "yellow"]
# Positions: 0, 1, 2, 3

# The 16 symbols:
# Symbol("red0")     = "red is in position 0"
# Symbol("red1")     = "red is in position 1"
# Symbol("blue0")    = "blue is in position 0"
# ... and so on for all combinations

# The answer we are looking for is something like:
# red is in position 2, blue is in position 0, etc.
```

7.2 Structural Constraints — The Game Rules

Before adding any guesses, we encode the rules of Mastermind itself:

Rule A — Every color must appear somewhere:

```
# For each color, it must be in position 0, 1, 2, or 3
for color in colors:
    knowledge.add(Or(
```

```
        Symbol(f"{color}0"),
        Symbol(f"{color}1"),
        Symbol(f"{color}2"),
        Symbol(f"{color}3")
    ))
# In plain English: "red must be somewhere, blue must be somewhere, ..."
```

Rule B — Each color appears in exactly one position:

```
# If red is in position 0, it cannot ALSO be in position 1, 2, or 3
for color in colors:
    for i in range(4):
        for j in range(4):
            if i != j:
                knowledge.add(Implication(
                    Symbol(f"{color}{i}"), Not(Symbol(f"{color}{j}"))
                ))
# In plain English: "If red is in pos 0, then red is NOT in pos 1" etc.
```

Rule C — Each position holds exactly one color:

```
# If red is in position 0, then blue/green/yellow cannot be in position 0
for i in range(4):
    for c1 in colors:
        for c2 in colors:
            if c1 != c2:
                knowledge.add(Implication(
                    Symbol(f"{c1}{i}"), Not(Symbol(f"{c2}{i}"))
                ))
# In plain English: "If red is in pos 0, then blue is NOT in pos 0" etc.
```

7.3 Encoding Guess Feedback — The Tricky Part

Suppose we guessed `[red in pos 0, blue in pos 1, green in pos 2, yellow in pos 3]` and the feedback was: **exactly 2 of these 4 positions are correct**.

Why is this tricky?

We don't know **which** 2 positions are correct! So we must say: "Either positions 0 and 1 are the correct pair, OR positions 0 and 2, OR positions 0 and 3, OR..." We enumerate all $\binom{4}{2} = 6$ possible pairs and wrap them in a big OR.

For each pair, we also explicitly say which positions are **WRONG** using Not(). This

is critical — without the Not()s, a world where all 4 are correct would also satisfy the Or, contradicting the "exactly 2" feedback.

```
# Feedback: EXACTLY 2 of {red0, blue1, green2, yellow3} are correct
# There are C(4,2) = 6 possible pairs:
knowledge.add(Or(
    And(Symbol("red0"), Symbol("blue1"),
        Not(Symbol("green2")), Not(Symbol("yellow3"))),    # pair: pos0 + pos1

    And(Symbol("red0"), Symbol("green2"),
        Not(Symbol("blue1")), Not(Symbol("yellow3"))),     # pair: pos0 + pos2

    And(Symbol("red0"), Symbol("yellow3"),
        Not(Symbol("blue1")), Not(Symbol("green2"))),      # pair: pos0 + pos3

    And(Symbol("blue1"), Symbol("green2"),
        Not(Symbol("red0")), Not(Symbol("yellow3"))),      # pair: pos1 + pos2

    And(Symbol("blue1"), Symbol("yellow3"),
        Not(Symbol("red0")), Not(Symbol("green2"))),       # pair: pos1 + pos3

    And(Symbol("green2"), Symbol("yellow3"),
        Not(Symbol("red0")), Not(Symbol("blue1")))         # pair: pos2 + pos3
))
```

Feedback 2 — "None of these specific colors are in this specific wrong position":

```
# From game feedback: we learned these specific placements are wrong
knowledge.add(And(
    Not(Symbol("blue0")),       # blue is NOT in position 0
    Not(Symbol("red1")),        # red is NOT in position 1
    Not(Symbol("green2")),      # green is NOT in position 2 (already knew this from feedback 1)
    Not(Symbol("yellow3"))      # yellow is NOT in position 3
))
```

8. puzzle.py — Harry Potter House Assignment

What This Program Does

Four wizards must each be assigned to exactly one Hogwarts house. Using a few clues, the AI figures out who belongs where. This is a classic **constraint satisfaction puzzle**, similar to the famous Zebra puzzle or Sudoku.

8.1 Variables

```
people = ["Gilderoy", "Pomona", "Minerva", "Horace"]
houses = ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"
    ]

# 16 symbols total, one for each (person, house) pair:
# Symbol("GilderoyGryffindor")  means: "Gilderoy is in
    Gryffindor"
# Symbol("PomnaHufflepuff")     means: "Pomona is in
    Hufflepuff"
# ...and so on for all 16 combinations
```

8.2 Constraints and Clues

Constraint	Code Pattern	Plain English
Each person in ≥ 1 house	`knowledge.add(Or(...)`	Every person must be assigned somewhere
No person in > 1 house	`Implication(PH1, Not(PH2))`	If Gilderoy is in Gryffindor, he is NOT also in Hufflepuff
No house has > 1 person	`Implication(P1H, Not(P2H))`	If Gilderoy is in Gryffindor, Pomona is NOT in Gryffindor
Gilderoy: Gryffindor or Ravenclaw	`Or(GG, GR)`	Clue from puzzle
Pomona: not Slytherin	`Not(PS)`	Clue from puzzle
Minerva: in Gryffindor	`Symbol("MinervaGryffi`	Direct fact

8.3 Full Deduction Chain — Step by Step

Follow the Reasoning

Step 1: Clue says Minerva = Gryffindor. ⇒ **Minerva: Gryffindor** (direct).

Step 2: One-person-per-house rule: since Minerva is in Gryffindor, no one else can be. ⇒ Gilderoy is **NOT** in Gryffindor.

Step 3: Gilderoy's clue says Gryffindor OR Ravenclaw. Gryffindor is blocked. ⇒ **Gilderoy: Ravenclaw**.

Step 4: Ravenclaw is now taken. Both Horace and Pomona are NOT in Gryffindor or Ravenclaw. They must be in Hufflepuff or Slytherin.

Step 5: Pomona's clue says NOT Slytherin. So Pomona must be Hufflepuff. ⇒ **Pomona: Hufflepuff**.

Step 6: Only Slytherin remains. ⇒ **Horace: Slytherin**.

1. Clue: Minerva is in Gryffindor (direct fact).
2. One-person-per-house rule: Gilderoy is **NOT** in Gryffindor.
3. Gilderoy's clue: `Or(Gryffindor, Ravenclaw)` ⇒ Gilderoy must be in **Ravenclaw**.
4. One-person-per-house: Horace and Pomona are NOT in Gryffindor or Ravenclaw.
5. Pomona's clue: `Not(Slytherin)` ⇒ Pomona must be in **Hufflepuff**.
6. Only Slytherin remains for **Horace**.

Person	House	How we know
Minerva	Gryffindor	Direct clue
Gilderoy	Ravenclaw	Gryffindor blocked + Or clue
Pomona	Hufflepuff	Ravenclaw/Gryffindor blocked + Not(Slytherin)
Horace	Slytherin	Only remaining house

9. Lab Exercises

Complete the following exercises in order. For each exercise, show your code, the output, and a brief explanation of what the result means.

9.1 Exercise 1 — Truth Table Analysis (Warm-up)

Without running any code, fill in the truth table for:

```
Implication(And(P, Q), Or(P, Not(Q)))
```

How to read this sentence

In plain English: "**If** (P is true AND Q is true), **then** (P is true OR Q is false)."

To fill in the table, use the rules you learned:
– Compute And(P,Q) first. Then compute Or(P, Not(Q)). Then apply Implication.

P	Q	$P \wedge Q$	$P \vee \neg Q$	$(P \wedge Q) \rightarrow (P \vee \neg Q)$
T	T	T	T	?
T	F	F	T	?
F	T	F	F	?
F	F	F	T	?

Then verify your answers using `model_check()`.

9.2 Exercise 2 — Extend `harry.py`

Add a fourth character: Snape. Encode the following additional rules, then check what conclusions can be drawn:

- Harry met exactly one of: Hagrid, Dumbledore, or Snape (update the `Or`)
- If it is raining, Snape did not come outside
- Snape was seen outdoors today (add this as a new fact)

Hint

Start from the existing `harry.py` code. Add `snape = Symbol("snape")` and update the `Or` and `Not(And(...))` constraints to include Snape. Then add the two new rules as `Implication` and a direct `Symbol` fact.

Write the full modified knowledge base and report all conclusions `model_check()` can prove.

9.3 Exercise 3 — New Clue Scenario

Modify `clue.py` with a completely new set of characters:

- Suspects: `AliceSuspect`, `BobSuspect`, `CarolSuspect`
- Rooms: `office`, `garage`, `basement`
- Weapons: `poison`, `rope`, `candlestick`

Encode the following clues:

1. Cards in your hand: `Not(alice)`, `Not(office)`, `Not(poison)`
2. Opponent showed you one of: `Not(bob)`, `Not(garage)`, `Not(rope)`
3. You separately learned: `Not(candlestick)`, `Not(basement)`

Hint

Don't forget Layer 1: add the initial Or constraints (at least one suspect, room, weapon). Layer 2 uses And(Not(...), Not(...), ...). Layer 3 uses Or(Not(...), Not(...), ...). Layer 4 uses individual Not().

Run `check_knowledge()` and report which items are YES, MAYBE, and implicitly NO.

9.4 Exercise 4 — Add a Third Clue to `mastermind.py`

Suppose a third guess `[blue in pos 0, green in pos 1, red in pos 2, yellow in pos 3]` receives the feedback: **0 correct positions**. Add this as a new `knowledge.add()` statement.

Hint

"0 correct positions" means each of the four guessed placements is **wrong**. This is simple to encode: `And(Not(Symbol("blue0")), Not(Symbol("green1")), Not(Symbol("red2")), Not(Symbol("yellow3")))`.

Does this allow `model_check` to completely solve the puzzle? Report your findings and explain why or why not.

9.5 Exercise 5 — Scalability Analysis (Discussion)

Answer the following questions in your lab report:

1. How many symbols does `puzzle.py` use? How many truth-table rows does `model_check` explore? (Formula: 2^n where n = number of symbols)
2. How many symbols does `mastermind.py` use? How many rows?
3. If we extended `mastermind.py` to 6 colors and 6 positions, how many symbols would there be? How many rows?
4. What algorithmic approach could replace truth-table enumeration for large problems? Name one and briefly explain how it improves on the exponential complexity.

Hint for Question 4

Look up "SAT solvers" or "DPLL algorithm". These use techniques like *unit propagation* (if only one variable can satisfy a clause, assign it immediately) and *backtracking* (undo bad decisions early) to avoid exploring all 2^n rows.

10. Summary

What You Built in This Lab

You built a **logical reasoning engine** from scratch. Starting from simple True/False variables (`Symbol`), you composed them into complex rules using connectives (`Not`, `And`, `Or`, `Implication`, `Biconditional`), collected those rules into a knowledge base, and then used `model_check()` to ask the AI: "*Does this fact follow necessarily from everything I know?*"

This is different from machine learning. There is no training data. There are no probabilities. Just **pure logic**: if the KB is true, then the conclusion is guaranteed.

Concept	How it appeared in this lab
Propositional variables	Rain, suspects, colors, house assignments
Logical connectives	`Not`, `And`, `Or`, `Implication`, `Biconditional` classes
Knowledge base	`And()` object built up with `.add()` calls
Model checking	`model_check()` performing truth-table enumeration
Entailment vs probability	YES = logically certain; MAYBE = underdetermined
Constraint encoding	Structural rules + puzzle clues as logical sentences
Scalability limit	2^n enumeration becomes infeasible for large n

Connection to Broader AI

The `model_check()` approach is called **Model Checking** or **Truth-Table Enumeration**. Real-world systems like SAT solvers and the DPLL algorithm extend this idea with backtracking, unit propagation, and heuristic search to handle thousands of variables.

Knowledge-based AI with logical inference is foundational to **expert systems**, **automated theorem provers**, and **formal verification** of software and hardware.